

Trabajo Practico N° 2

1. Algoritmos de Scheduling

La planificación (Scheduling) de CPU se ocupa del problema de decidir cual de los procesos que están en la cola de listos debe recibir la CPU.

Para ello existen diferentes tipos de algoritmos de Scheduling:

• Algoritmo FIFO (First Come First Served)

Cuando hay que elegir un proceso para ejecutarse, se selecciona el más antiguo. No favorece a ningún tipo de procesos, pero en principio se podría decir que los CPU-Bound terminan al comenzar su primera ráfaga, mientras que los I/O-Bound requieren múltiples ráfagas

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1	2	3	4	5	6<										6	0
2	1	3		>					1	2	3<							8	5
3	2	2			>							1	2<					9	7
4	3	4				>								1	2	3	4<	12	8
																		35	20

Adecuado para

- Sistemas por lotes (batch) donde los procesos no requieren interacción.
- Cuando no importa la rapidez de respuesta, sino la simplicidad.

Ventajas y desventajas

- Fácil de implementar.
- Es justo en el sentido de “primero en llegar, primero en servir”.
- No apropiativo, un proceso largo bloquea a los demás.
- Efecto convoy, los procesos cortos esperan mucho si hay uno largo antes.

• Algoritmo SJF (Shortest Job First)

Política non preemptive que selecciona el proceso con la ráfaga más corta.

Cálculo basado en la ejecución previa.

Procesos cortos se colocan delante de procesos largos (se va ordenando así la cola de listos).

Los procesos largos pueden sufrir starvation (inanición)

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1	2	3	4	5	6<										6	0
2	1	3		>							1	2	3<					10	7
3	2	2			>				1	2<								6	4
4	3	4				>								1	2	3	4<	12	8
																		34	19

Adecuado para

- Sistemas por lotes donde se conocen los tiempos de ejecución.
- No recomendable en sistemas interactivos o tiempo real.

Ventajas y desventajas

- Minimiza el tiempo promedio de espera.
- Puede causar inanición (procesos largos esperan indefinidamente).
- Requiere conocer o estimar el burst time.

• Algoritmo SRTF (Shortest Remaining Time First)

Versión apropiativa (preemptive) del SJF, selecciona el proceso con menos tiempo de ejecución es su siguiente rafaga.

Favorece a los procesos I/O-Bound

Si llega un proceso con menor tiempo de ejecución que el actual, este se adueña del CPU

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1										2	3	4	5	6<	15	9
2	1	3		>1			2	3<										5	2
3	2	2			>1	2<												2	0
4	3	4				>			1	2	3	4<						6	2
Queue: P1 P2 P3 P2 P4 P1																		28	13

Adecuado para

- Sistemas por lotes donde se conocen los tiempos de ejecución.
- No recomendable en sistemas interactivos o tiempo real.

Ventajas y desventajas

- Minimiza el tiempo promedio de espera.
- Puede causar inanición (procesos largos esperan indefinidamente).
- Requiere conocer o estimar el burst time.

• Algoritmo RR (Round Robin)

Cada proceso recibe un quantum (intervalo de tiempo fijo).

Cuando su tiempo se acaba, pasa al final de cola y se ejecuta el siguiente.

Ejemplo con Q=2

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1	2			3	4						5	6<			13	7
2	1	3		>	1	2							3<					10	7
3	2	2			>				1	2<								6	4
4	3	4				>					1	2				3	4<	12	8
Queue: P1 P2 P1 P3 P4 P2 P1 P4																		41	26

Adecuado para

- Sistemas interactivos o multitarea.
- Ideal cuando se busca equidad y respuesta rápida.

Ventajas y desventajas

- Justo con todos los procesos.
- Buen tiempo de respuesta.
- Si el quantum es muy grande, se comporta como FCFS.
- Si el quantum es muy pequeño hay muchas interrupciones (**overhead**).

• Algoritmo con uso de Prioridades

Cada proceso tiene un valor que representa su prioridad -> menor valor mayor prioridad. Se selecciona el proceso de menor prioridad que se encuentran en la Ready Queue.

Existe una Ready Queue por cada nivel de prioridad.

Puede ser apropiativo o no.

Ejemplo no apropiativo

JOB	LLEG	CPU	PRI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	4	>1	2	3	4	5	6<						1	2	3<		6	0
2	1	3	2		>														12	9
3	2	2	3			>											1	2<	13	11
4	3	4	1				>			1	2	3	4<						7	3
Queue 1: P4																			38	23
Queue 2: P2																				
Queue 3: P3																				
Queue 4: P1																				

Ejemplo apropiativo

JOB	LLEG	CPU	PRI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	4	>1										2	3	4	5	6<	14	8
2	1	3	2		>1	2					3<								7	4
3	2	2	3			>						1	2<						8	6
4	3	4	1				>1	2	3	4<									4	0
Queue 1: P4																			33	18
Queue 2: P2 P2																				
Queue 3: P3																				
Queue 4: P1 P1																				

Adecuado para

- Sistemas en tiempo real o críticos, donde algunos procesos son más urgentes.

Ventajas y desventajas

- Permite controlar qué procesos son más importantes.
- Flexible para sistemas con diferentes niveles de urgencia.
- Puede causar inanición de procesos de baja prioridad.
- Requiere gestión cuidadosa de prioridades.

• Algoritmo con uso de Recursos

Cada proceso puede tener uno o más recursos de entrada salida.

El instante del I/O es el instante del proceso, no de los instantes generales.

Cuando un proceso ejecuta un recurso los demás procesos pueden seguir ejecutándose.

Ejemplo FCFS con recursos

JO B	LLE G	CP U	I	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	(R1,2,1)	>1	2	R1			3	4	5	6<							9	3
2	1	3	(R2,2,4)		>	1	2	R2	R2						3<				11	8
3	2	2	(R2,1,1)			>		1	CR2	R2						2<			11	9
4	3	4	(R3,2,1)				>						1	2	R3		3	4<	12	8
Ready Queue: P1 P2 P3 P1 P4 P2 P3																			43	27
Queue R1: P2																				
Queue R2: P3																				
Queue R3: P1																				

Criterios de desempate de los Algoritmos de Planificación

- Orden de llegada del proceso
- PID del proceso

Siempre se mantiene la misma política.

- CPU (Tcpu): tiempo que efectivamente usa la CPU el proceso.
- **Retorno (Tr):** tiempo que transcurre entre que el proceso llega al sistema hasta que completa su ejecución.
- **Espera (Te):** tiempo que el proceso se encuentra en el sistema esperando, es decir el tiempo que pasa sin ejecutarse. $Te = (Tr - Tcpu)$
- **Promedios (TPR y TPE):** tiempos promedio de Retorno y Espera. Promedio calculado de los tiempos individuales de cada proceso del lote.

2. Ejercicio procesos

Ejercicio 2:

<https://drive.google.com/drive/folders/1c3bokqNNd2qaDKD7fZzroq5Q-Gxt02z8?usp=sharing>

3. Ejercicio procesos

Ejercicio 3:

https://drive.google.com/drive/folders/1RTNJQ1z9JHX6D1Fg_7jnlvXAJJv1luRW?usp=sharing

4. Ejercicio procesos

Ejercicio 4:

<https://drive.google.com/drive/folders/1HubNrMvAk8qlzHvo7oDP91dnGHuCUEPM?usp=sharing>

5. Ejercicio procesos

Ejercicio 5:

https://drive.google.com/drive/folders/13AXKTt-oCpla6DMRnJava5xlv0WvD_AA?usp=sharing

6. Inanición (Starvation) y Envejecimiento (Aging)

a. La **inanición** es una situación en la que un proceso espera indefinidamente para obtener la CPU porque siempre hay otros procesos que tienen mayor prioridad o son elegidos antes.

Es decir, el proceso no está bloqueado ni termina, pero puede pasar un tiempo indefinido sin ejecutarse.

b. Algoritmos que pueden causar inanición.

- Prioridades** -> Si continuamente llegan procesos con prioridad más alta.
- SJF** -> Si continuamente llegan procesos con ráfagas más cortas.
- SRTF** -> Un proceso largo puede ser interrumpido por procesos más cortos.

Los que no pueden causar inanición.

- FCFS**
- Round Robin**

c. La técnica para evitar la inanición es llamada **Envejecimiento (Aging)**.

Consiste en aumentar la prioridad de los procesos que llevan mucho tiempo esperando.

Ej: un Aging de 4 unidades.

Un proceso con prioridad 3 está en su cola correspondiente, lo que se hace es que pasadas las 4 unidades de tiempo el proceso se sube a la cola de prioridad 2, Si en esta cola es atendido retorna a su cola original, en caso contrario pasados otras 4 unidades de tiempo el proceso se mueve a la cola de prioridad 1. Esto se repite hasta que el proceso obtiene la CPU.

7. Ejercicio procesos

Ejercicio 7:

https://drive.google.com/drive/folders/1taXWUs7JJ7B2eNjE9bL_Lhz748rM8l1e?usp=sharing

8. Procesos ligados a CPU y procesos ligados a E/S

- **Procesos ligados a CPU (CPU-bound):** realizan mucho cálculo y pocas operaciones de E/S.
- **Procesos ligados a E/S (I/O-bound):** ejecutan poca CPU y frecuentemente se bloquean esperando entrada/salida.

a. Round Robin

Ventaja:

- Es un algoritmo muy adecuado cuando hay procesos ligados a E/S.
- Como el quantum es pequeño, los procesos de E/S suelen ejecutar un poco de CPU y luego bloquearse para hacer la operación de entrada/salida, liberando rápidamente la CPU para otros procesos.
- Todos los procesos reciben CPU periódicamente, por lo que hay buena interactividad.

Desventaja:

- Si hay muchos procesos ligados a CPU, se producen muchos cambios de contexto al agotarse el quantum.
- Esto genera una sobrecarga (overhead) que disminuye el rendimiento del sistema, especialmente si el quantum es muy pequeño.

Round Robin favorece a los procesos ligados a E/S, ya que obtienen CPU rápidamente y suelen bloquearse antes de consumir todo el quantum. Sin embargo, con muchos procesos ligados a CPU puede generar un gran número de cambios de contexto, reduciendo la eficiencia.

b. SRTF (Shortest Remaining Time First)

Ventaja:

- Los procesos ligados a E/S suelen tener ráfagas de CPU muy cortas.
- Como SRTF elige el proceso con menor tiempo restante, estos son favorecidos

Desventaja:

- Los procesos ligados a CPU suelen tener ráfagas largas.
- Pueden ser interrumpidos continuamente por la llegada de procesos con ráfagas más cortas, demorando mucho su finalización e incluso pudiendo sufrir **inanición (starvation)**.

En SRTF los procesos ligados a E/S son favorecidos porque poseen ráfagas de CPU cortas. En cambio, los procesos ligados a CPU pueden ser postergados repetidamente por la llegada de procesos más cortos, aumentando su tiempo de espera e incluso pudiendo sufrir inanición.

9. Ejercicio procesos

Ejercicio 9:

<https://drive.google.com/drive/folders/1Toa0fXQCkKAkN3JKv6KBMBV1Qb9dh8uU?usp=sharing>

10. Pregunta conceptual VRR

Sí, puede suceder que el quantum de un proceso nunca llegue a cero. Si un proceso se bloquea para realizar una operación de entrada/salida **antes de que ocurra dicha interrupción**, el contador no disminuye y el proceso conserva todo el quantum restante. En **Virtual Round Robin (VRR)**, cuando ese proceso finaliza la E/S y vuelve al estado listo, **recupera el quantum que no consumió** y pasa primero por la cola auxiliar para utilizarlo. Si este comportamiento se repite en cada ejecución (es decir, el proceso siempre se bloquea antes del siguiente tick del reloj), **el contador del quantum nunca se decrementará**, por lo que **el quantum nunca llegará a cero**.

11. Estimación de la duración de las ráfagas de CPU

El principal problema de SJF y STRF es que necesitan conocer la duración de la próxima ráfaga de CPU, pero el sistema operativo no puede conocer el futuro, por eso utiliza las estimaciones basadas en el comportamiento anterior del proceso.

$T(i)$ = duración de la ráfaga de CPU i -ésima del proceso

$S(i)$ = valor estimado para el i -ésimo caso

$S(1)$ = valor estimado para la primera ráfaga de CPU. No es calculado

Proceso con las siguientes ráfagas:

6, 4, 6, 4, 13, 13, 13

Valor inicial estimado de $S(1) = 10$

a. Usando la siguiente formula:
$$S_{n+1} = \frac{1}{n}T_n + \frac{n-1}{n}S_n$$

Cálculos:

Estimación	Cálculo	Resultado
S_1	Dato	inicial 10
S_2	6	6
S_3	$(6+4) / 2$	5
S_4	$(6+4+6) / 3$	5,33
S_5	$(6+4+6+4) / 4$	5
S_6	$(6+4+6+4+13) / 5$	6,6
S_7	$(6+4+6+4+13+13) / 6$	7,67

b. Usando la siguiente formula: $S_{n+1} = \alpha T_n + (1 - \alpha)S_n$

- Si α es cercano a 1, la estimación depende principalmente de la última ráfaga, por lo que se adapta rápidamente a los cambios.
- Si α es cercano a 0, la estimación depende más de las estimaciones anteriores y cambia lentamente.

Los casos más recientes tienen mayor influencia cuando α toma valores cercanos a 1

c. Calculo de estimaciones con la formula: $S_{n+1} = \alpha T_n + (1 - \alpha)S_n$

$\alpha = 0,2$

<u>Estimación</u>	<u>Valor</u>
S_1	10
S_2	9,2
S_3	8,16
S_4	7,73
S_5	6,98
S_6	8,18
S_7	9,14

$\alpha = 0,5$

<u>Estimación</u>	<u>Valor</u>
S_1	10
S_2	8
S_3	6
S_4	6
S_5	5
S_6	9
S_7	11

$\alpha = 0,8$

<u>Estimación</u>	<u>Valor</u>
S_1	10
S_2	6,8
S_3	4,56
S_4	5,71
S_5	4,34
S_6	11,27
S_7	12,65

d. Las ráfagas reales son:

6, 4, 6, 4, 13, 13, 13

Se observa que el proceso cambia bruscamente de ráfagas cortas a largas.

La media simple tarda mucho en reflejar ese cambio.

Con $\alpha = 0,2$ la estimación también cambia lentamente.

Con $\alpha = 0,5$ la adaptación es mejor.

Con $\alpha = 0,8$ la estimación responde rápidamente al aumento de las ráfagas.

La estimación con $\alpha = 0,8$ es la que más se asemeja a las ráfagas reales, ya que otorga mayor peso a las ráfagas más recientes y se adapta rápidamente a los cambios en el comportamiento del proceso.

12. Colas Multinivel

a). Tenemos dos tipos de procesos:

Procesos interactivos

Son procesos como:

- navegador
- editor de texto
- terminal
- aplicaciones gráficas

Lo más importante es que **respondan rápido al usuario**.

Por eso conviene usar: **Round Robin (RR)**

Porque reparte la CPU entre todos los procesos y ninguno espera demasiado tiempo para ejecutarse. Además, los procesos interactivos suelen ser **I/O-bound**, es decir, ejecutan poca CPU y luego esperan entrada/salida, por lo que RR funciona muy bien.

Procesos Batch

Son procesos que se ejecutan sin interacción con el usuario.

Ejemplos:

- compilar un programa grande
- realizar cálculos científicos
- procesamiento de datos

Lo importante no es responder rápido sino terminar el trabajo eficientemente.

Una buena opción es: **FCFS**

Porque es simple y evita cambios de contexto innecesarios.

También podría utilizarse **SJF**, ya que minimiza el tiempo de espera promedio, aunque requiere estimar las ráfagas de CPU.

b) Para planificar las colas utilizaría un algoritmo por prioridades, asignando mayor prioridad a la cola de procesos interactivos y menor prioridad a la cola de procesos Batch. De esta manera, los procesos interactivos obtienen una respuesta rápida, mientras que los Batch utilizan la CPU cuando la cola interactiva está vacía.

Ejemplo:

Tengo 2 Ready queue:

- Cola 1: Es usada para los procesos interactivos, con mayor prioridad. Esta cola se administra con Round Robin (por ejemplo, con quantum 2)
- Cola 2: Es usada para procesos Batch, con menor prioridad. Esta cola se administra con FCFS.

13. Ejercicio procesos

Ejercicio 13:

<https://drive.google.com/drive/folders/16Eou0ILy6q1Gs76HU0fl1XsAlOpMcKn6?usp=sharing>

14. Ejercicio procesos

Ejercicio 14:

<https://drive.google.com/drive/folders/1oBTrMwejeGV1KDmEY7CGD6F4ibHIH0u6?usp=sharing>

15. Colas Multinivel con Realimentación

Implementaría tres colas con realimentación. Todos los procesos ingresarían inicialmente a la cola de mayor prioridad (Q1), administrada mediante Round Robin con un quantum pequeño (por ejemplo, 2). Si un proceso consume completamente su quantum, se considera que requiere mucho tiempo de CPU y desciende a la cola Q2, también administrada mediante Round Robin, pero con un quantum mayor (por ejemplo, 4). Si nuevamente consume todo el quantum, desciende a la cola Q3, administrada mediante FCFS.

Las colas se administrarían mediante prioridades, dando preferencia a las colas superiores. De esta forma, los procesos interactivos, que suelen bloquearse por E/S antes de agotar el quantum, permanecen en las colas de mayor prioridad, mientras que los procesos CPU-bound son penalizados descendiendo de cola.

Para evitar la inanición, aplicaría Aging (envejecimiento), incrementando la prioridad de los procesos que permanezcan mucho tiempo esperando en las colas inferiores, permitiendo que eventualmente vuelvan a una cola superior y obtengan la CPU.

16. Estrategias de administración

- **Cortos acotados por CPU:** usan poca CPU y terminan rápido.
- **Cortos acotados por E/S:** usan poca CPU, hacen muchas operaciones de E/S y terminan rápido.
- **Largos acotados por CPU:** usan mucha CPU y tardan mucho en finalizar.
- **Largos acotados por E/S:** duran bastante tiempo porque realizan muchas operaciones de E/S, aunque cada ráfaga de CPU suele ser corta.

a) Prioridad estática con SJF

Siempre ejecuta primero el proceso cuya **próxima ráfaga de CPU es la más corta**.
Entonces:

Beneficia

- **Cortos acotados por CPU** -> Como la ráfaga es corta, termina rápidamente.
- **Cortos acotados por E/S** -> También ráfagas cortas.

Perjudica

- **Largos acotados por CPU** -> Ráfagas largas, por lo que esperan mucho.
- **Largos acotados por E/S** -> Ráfagas cortas, pero trabajo completo largo.

b) **Prioridad dinámica inversamente proporcional al tiempo desde la última E/S** significa:

- Si un proceso **hizo una E/S hace muy poco**, tiene **alta prioridad**.
- Si hace mucho que no realiza E/S, su **prioridad disminuye**.

Los procesos **I/O-bound** hacen muchas operaciones de E/S, por lo tanto, el algoritmo favorece a esos procesos.

Beneficia

- **Cortos acotados por E/S.**
- **Largos acotados por E/S.**

Porque realizan E/S frecuentemente y recuperan prioridad cada que vuelven de ella.

Perjudica

- **Cortos acotados por CPU.**
- **Largos acotados por CPU.**

Porque permanecen mucho tiempo usando la CPU sin realizar E/S, por lo que su prioridad disminuye.

17. Round Robin = FIFO

En Round Robin cada proceso puede ejecutar como máximo durante un **quantum**. Si el quantum aumenta sin límite, llegará un momento en que será mayor que el tiempo de CPU que necesitan la mayoría de los procesos. En consecuencia, los procesos finalizarán su ejecución antes de agotar el quantum y no serán reencolados. Por lo tanto, cada proceso ejecutará la CPU hasta terminar, respetando el orden de llegada, exactamente como ocurre en el algoritmo **FIFO (FCFS)**. Por ello, cuando el quantum tiende a infinito, **Round Robin se aproxima al comportamiento de FIFO**.

18. Fork

a.

```
printf("hola")
x = fork()
if x < 1 {
    execv("ls")
    printf("mundo")
    exit(0)
}
exit(0)
```

- Imprime:

hola

salida del comando ls -> hijo

b.

```
printf("hola")
x = fork()
if x < 1 {
    execv("ps")
    printf("mundo")
    exit(0)
}
execv("ls")
printf("fin")
exit(0)
```

- Imprime:

hola

salida del comando ps -> hijo

salida del comando ls -> padre

C.

```
printf("Anda a rendir el Primer Parcial de
Promo!")
newpid = fork()
if newpid == 0 {
    printf("Estoy comenzando el Examen")
    execv("ps")
    printf("Termine el Examen")
}
printf("¿Como te fue?")
exit(0)
printf("Ahora anda a descansar")
```

- Imprime:
Anda a rendir el Primer Parcial de Promo!
Estoy comenzando el Examen -> hijo
salida del comando ps -> hijo
¿Como te fue? -> padre

19. Fork en C

a. Inicio 1 proceso -> fork 2 procesos -> fork 4 procesos -> fork 8 procesos
Aparecen 8 líneas con la palabra "Proceso"

b. Si porque cada proceso se ejecuta exactamente una vez.

20. Fork en C 2

Cuando un proceso hace fork(), el hijo recibe una copia de toda la memoria del padre.
A partir de ese momento, **cada proceso tiene su propia copia de las variables.** Si un proceso modifica una variable, **los demás no se ven afectados.**

a. Todos imprimirán: Proceso 1
y se imprimen 8 líneas.

b. Si porque todos empiezan en 0 y todos hacen exactamente un p++

c. El único valor que aparece es 1.

d. Si pongo el p++ dentro del for y antes del fork, el valor heredado por los proceso cambia. Por ejemplo, con 3 iteraciones todos los procesos terminarían imprimiendo 3, porque el incremento se realiza antes de cada fork y los hijos heredan el valor actualizado.

21. Segmentación

Dado un esquema de segmentación donde cada dirección hace referencia a 1 byte y la siguiente tabla de segmentos de un proceso, traduzca, de corresponder, las direcciones lógicas indicadas a direcciones físicas. La Dirección lógica está representada por: **segmento:desplazamiento**

Segmento	Dir. Base	Tamaño
0	102	12500
1	28699	24300
2	68010	15855
3	80001	400

- i. 0000:9001
- ii. 0001:24301
- iii. 0002:5678
- iv. 0001:18976
- v. 0003:0

i. 0000:9001 -> segmento 0, desplazamiento 9001

Validación: Desplazamiento < tamaño = 9001 < 12500 **dirección valida ✓**

Dirección física = base + desplazamiento = 102 + 9001 = 9103

ii. 0001:24301 -> segmento 1, desplazamiento 24301

Validación: Desplazamiento < tamaño = 24301 >= 24300 **✗**

Dirección invalida

iii. 0002:5678 -> segmento 2, desplazamiento 5678

Validación: Desplazamiento < tamaño = 5678 < 15855 **dirección valida ✓**

Dirección física = base + desplazamiento = 68010 + 5678 = 73688

iv. 0001:18976 -> segmento 1, desplazamiento 18976

Validación: Desplazamiento < tamaño = 18976 < 24300 **dirección valida ✓**

Dirección física = base + desplazamiento = 28699 + 18976 = 47675

v. 0003:0 -> segmento 3, desplazamiento 0

Validación: Desplazamiento < tamaño = 0 < 400 **dirección valida ✓**

Dirección física = base + desplazamiento = 0 + 80001 = 80001

🔔 22. Paginación

- Tamaño de página 512 bytes
- Cada dirección referencia 1 byte
- Memoria Principal de 10240 bytes (20 KiB)

Proceso P1 -> tamaño lógico 2000 bytes con la siguiente tabla de páginas:

Página	Marco
0	3
1	5
2	2
3	6

Marco	Inicio-Fin
2	1024 - 1535
3	1536 - 2047
5	2560 - 3071
6	3072 - 3583

a. Cantidad de marcos = Memoria Principal / Tamaño de pagina = $10240 / 512 = 20$ (0..19)

Cantidad de paginas = Tamaño P1 / Tamaño de pagina = $2000 / 512 = 3,9 \rightarrow 4$ (0..3)

Inicio = Marco * tamaño de pagina

Fin = (Marco + Tamaño de página) - 1

b. Indicar si las siguientes direcciones lógicas corresponden al espacio lógico del proceso P1 y en caso afirmativo indicar la dirección física a la que corresponden:

i. 35 ii. 512 iii. 2051 iv. 0 v. 1325 vi. 602

Para validar si la dirección lógica corresponde al espacio lógico:

dirección lógica < tamaño lógico del proceso -> dir. lógica < 2000

i. 35 $35 < 2000$ **dirección valida** ✓

- Numero de página = $35 \text{ DIV } 512 = 0$
- Desplazamiento = $35 \text{ MOD } 512 = 35$
- Direccion Fisica = $(512 * 3) + 35 = 1571$

ii. 512 $512 < 2000$ **dirección valida** ✓

- Numero de página = $512 \text{ DIV } 512 = 1$
- Desplazamiento = $512 \text{ MOD } 512 = 0$
- Direccion Fisica = $(512 * 5) + 0 = 2560$

iii. 2051 ≥ 2000 **dirección invalida** ✗

iv. 0 $0 < 2000$ **dirección valida** ✓

- Numero de página = $0 \text{ DIV } 512 = 0$
- Desplazamiento = $0 \text{ MOD } 512 = 0$
- Direccion Fisica = $(512 * 3) + 0 = 1536$

v. 1325 $1325 < 2000$ **dirección valida ✓**

- Numero de página = $1325 \text{ DIV } 512 = 2$
- Desplazamiento = $1325 \text{ MOD } 512 = 301$
- Direccion Fisica = $(512 * 2) + 301 = 1325$

vi. 602 $602 < 2000$ **dirección valida ✓**

- Numero de página = $602 \text{ DIV } 512 = 1$
- Desplazamiento = $602 \text{ MOD } 512 = 90$
- Direccion Fisica = $(512 * 1) + 90 = 602$

c. Indicar, en caso de ser posible, las direcciones lógicas del proceso P1 que se corresponden a las siguientes direcciones físicas:

i. 509 ii. 1500 iii. 0 iv. 3215 v. 1024 vi. 2000

Para validar que la dirección física corresponde a el espacio lógico de P1:

El numero de marco calculado tiene que estar dentro de los marcos asignados a P1
osea: número de marco $\in \{2, 3, 5, 6\}$

i. 509 $0 \notin \{2, 3, 5, 6\}$ **dirección invalida ✗**

- Numero de marco = $509 \text{ DIV } 512 = 0$

ii. 1500 $2 \in \{2, 3, 5, 6\}$ **dirección valida ✓**

- Numero de marco = $1500 \text{ DIV } 512 = 2$
- Desplazamiento = $1500 \text{ MOD } 512 = 476$
- Dirección Logica = $(2 * 512) + 476 = 1500$

iii. 0 $0 \notin \{2, 3, 5, 6\}$ **dirección invalida ✗**

- Numero de marco = $0 \text{ DIV } 512 = 0$

iv. 3215 $6 \in \{2, 3, 5, 6\}$ **dirección valida ✓**

- Numero de marco = $3215 \text{ DIV } 512 = 6$
- Desplazamiento = $3215 \text{ MOD } 512 = 143$
- Dirección Logica = $(6 * 512) + 143 = 3139$

v. 1024 $2 \in \{2, 3, 5, 6\}$ **dirección valida ✓**

- Numero de marco = $1024 \text{ DIV } 512 = 2$
- Desplazamiento = $1024 \text{ MOD } 512 = 0$
- Dirección Logica = $(2 * 512) + 0 = 1024$

vi. 2000 $3 \in \{2, 3, 5, 6\}$ **dirección valida ✓**

- Numero de marco = $2000 \text{ DIV } 512 = 3$
- Desplazamiento = $2000 \text{ MOD } 512 = 464$
- Dirección Logica = $(3 * 512) + 464 = 1992$

23. Direcciones Físicas

Dado un esquema donde cada dirección hace referencia a 1 byte, con páginas de 2 KiB (KibiBytes), donde el frame 0 se encuentra en la dirección física 0. Con las siguientes primeras entradas de la tabla de páginas de un proceso

Página	Marco	
0	16	i. 5120
1	13	ii. 3242
2	9	iii. 1578
3	2	iv. 2048
4	0	v. 8191

traduzca las direcciones lógicas a físicas.

2 Kib = 2048 bytes

1 KiB = 2^{10} bytes $\rightarrow$ 2 Kib = 2×2^{10} bytes =

i. 5120

- Numero de Pagina = $5120 \text{ DIV } 2048 = 2$
- Desplazamiento = $5120 \text{ MOD } 2048 = 1024$
- Dirección física = $(9 * 2048) + 1024 = 19456$

ii. 3242

- Numero de Pagina = $3242 \text{ DIV } 2048 = 1$
- Desplazamiento = $3242 \text{ MOD } 2048 = 1194$
- Dirección física = $(13 * 2048) + 1194 = 27818$

iii. 1578

- Numero de Pagina = $1578 \text{ DIV } 2048 = 0$
- Desplazamiento = $1578 \text{ MOD } 2048 = 1578$
- Dirección física = $(16 * 2048) + 1578 = 34346$

iv. 2048

- Numero de Pagina = $2048 \text{ DIV } 2048 = 1$
- Desplazamiento = $2048 \text{ MOD } 2048 = 0$
- Dirección física = $(13 * 2048) + 0 = 26624$

v. 8191

- Numero de Pagina = $8191 \text{ DIV } 2048 = 4$
- Desplazamiento = $8191 \text{ MOD } 2048 = -1?$
- Dirección física = **dirección no valida?**

24. Esquema de paginación

Dado un esquema de paginación, donde cada dirección hace referencia a 1 byte, se tiene un tamaño de dirección de 32 bits y un tamaño de página de 2048 bytes. Suponga además un proceso P1 que necesita 51358 bytes para su código y 68131 bytes para sus datos.

- Cada dirección referencia a 1 byte
- Tamaño de dirección 32 bits $\rightarrow$ 4 bytes
- Tamaño de pagina de 2048 bytes $\rightarrow 2^{11}$ bytes
- P1 necesita 51358 bytes para su código y 68131 bytes para sus datos.
- Espacio de direcciones = $2^{\text{Tamaño de dirección}}$ $\rightarrow 2^{32}$ bytes

a. Una página tiene un tamaño de 2048 bytes $\rightarrow 2^{11}$ bytes y cada dirección tiene 32 bits

$2^{11} \rightarrow$ 11 bits para el desplazamiento

El resto para el numero de página $\rightarrow 32 - 11 = 21$ bits para el numero de pagina

b. Tamaño máximo de un proceso = $2^{\text{Espacio de direcciones}}$
= 2^{32} bytes

c. Cantidad máxima de paginas por proceso = $\text{Espacio de direcciones} / \text{Tamaño de páginas}$
= $2^{32} / 2^{11} = 2^{21}$ bytes

d. Ram de 4GiB $\rightarrow 1 \text{ GiB} = 2^{30}$ bytes $\rightarrow 4\text{GiB} = 2^{30} \times 4 = 2^{30} \times 2^2 = 2^{32}$ bytes
Tamaño de marco = Tamaño de página = 2^{11} bytes
Cantidad de marcos = $\text{Espacio disponible} / \text{Tamaño de marco}$
= $2^{32} / 2^{11} = 2^{21}$ bytes

e. Paginas necesarias = $\text{Tamaño del código} / \text{Tamaño de página}$
= $51358 / 2048 = 25,07 = 26 \rightarrow$ siempre redondeo para arriba

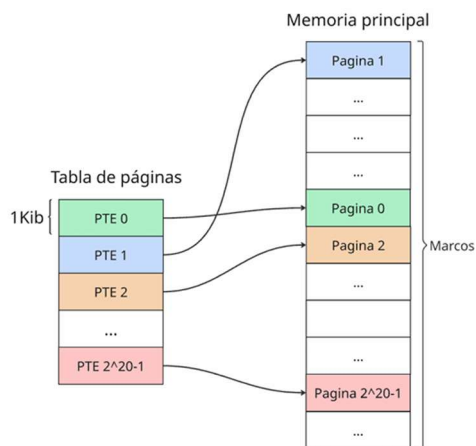
f. Paginas necesarias = $\text{Tamaño de los datos} / \text{Tamaño de pagina}$
= $38131 / 2048 = 33,26 = 34 \rightarrow$ siempre redondeo para arriba

g. código = 51358 bytes datos = 68131 bytes
Fragmentación interna = $\text{espacio asignado} - \text{espacio usado} =$
= $53248 - 51358 = 1890$ bytes de fragmentación interna para el código de P1
 $\text{espacio asignado} = \text{Paginas necesarias} \times \text{Tamaño de pagina} = 26 \times 2048$
= 53248

Fragmentación interna = $34 \times 2048 - 68131 = 69632 - 68131$
 = **1501 bytes de fragmentación interna para el código de P1**

Fragmentación interna total = $1890 + 1501 = 3391$ bytes

h. Tabla de paginas de 1 nivel osea:



En donde:

- 20 bits para identificar pagina
- 12 bits para identificar desplazamiento
- PTE de 1 KiB $\rightarrow 2^{10}$ bytes

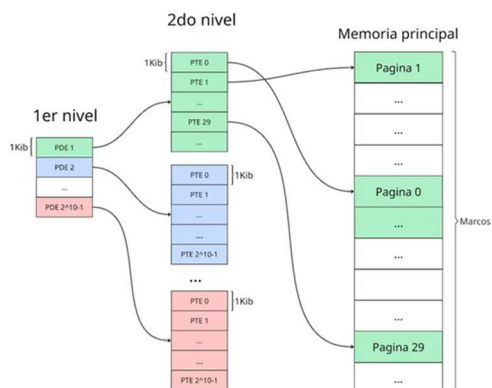
Buscamos saber el tamaño mínimo de la tabla de páginas del P1

Cantidad de paginas posibles = $2^{\text{bits para identificar pagina}}$
 = 2^{20}

1 nivel $\rightarrow 2^{20}$ páginas $\rightarrow 2^{20}$ entradas

Entonces 2^{20} entradas x Tamaño de PTE
 = $2^{20} \times 2^{10} = 2^{30}$ bytes

i. Tabla de páginas de 2 niveles osea:



10 bits para primer nivel $\rightarrow 2^{10}$ entradas
 10 bits para segundo nivel $\rightarrow 2^{10}$ entradas
 12 bits para desplazamiento $\rightarrow 2^{12}$ bytes
 $\rightarrow$ tamaño de página
 PTE 1KiB $\rightarrow 2^{10}$ bytes

-Cantidad de paginas de P1 para código = **13**

tamaño código / tamaño de página = $51358 / 2^{12} = 12,53 \rightarrow 13$

-Cantidad de paginas de P1 para datos = **17**

tamaño datos / tamaño de pagina = $68131 / 2^{12} = 15,63 \rightarrow 17$

Primer nivel de la tabla tiene 2^{10} entradas y cada PTE requiere 2^{10} bytes
 = 2^{20} bytes

Segundo nivel de la tabla tiene 2^{10} entradas y cada PTE requiere 2^{10} bytes
 = 2^{20} bytes

Entonces como tenemos que almacenar 30 paginas (13 + 17) con una sola tabla del segundo nivel nos va a alcanzar para almacenar las paginas del proceso 1. Pro lo tanto como mínimo va a ocupar:

Tabla del primer nivel + Tabla del segundo nivel = $2^{20} + 2^{20} = 2 \times 2^{20} \rightarrow 2^2 \times 2^{20}$
 = 2^{21} bytes

i. - **Tabla de un nivel** → ineficiente en cuanto espacio

- **Tabla de dos niveles** → más compleja de mantener, pero ahorra espacio. Además, requiere más accesos a memoria para poder resolver una dirección.

🐼 25. Segmentación Paginada

Dada la técnica de administración de memoria por medio de segmentación paginada y teniéndose disponibles las siguientes tablas:

Tabla de Segmentos		Tabla de Páginas		
Núm. Seg.	Dir. base	Nro. Segmento	Nro. Página	Direc. Base
1	500	1	1	40
			2	80
			3	60
2	1500	2	1	20
			2	25
			3	0
3	5000	3	1	120
			2	150

Indicar las

direcciones físicas correspondientes a las siguientes direcciones lógicas (segmento,página,desplazamiento):

i. (2,1,1)

ii. (1,3,15)

iii. (3,1,10)

iv. (2,3,5)

i. (2,1,1)

Segmento = 2

Numero de página = 1

Dirección Base = 20

Dirección física = Dirección base + Desplazamiento = $20 + 1 = 21$

ii. (1,3,15)

Segmento = 1 Numero de página = 3

Dirección Base = 60

Dirección física = $60 + 15 = 75$

iii. (3,1,10)

Segmento = 3 Numero de página = 1

Dirección Base = 120

Dirección física = $120 + 10 = 130$

iv. (2,3,5)

Segmento = 2 Numero de página = 3

Dirección Base = 0

Dirección física = $0 + 5 = 5$